💻 System configuration Chat Roles

Created	@16 de febrero de 2026 19:12
Module Code	56BA1A Desarrollo de aplicaciones con IA

Gestión de Roles y System Instructions en Gemini API

System Instruction

Para construir aplicaciones profesionales, la API de Gemini organiza la comunicación mediante roles y una configuración maestra llamada **System Instruction**.

▼ **El Poder de las System Instructions**

Las instrucciones del sistema te permiten "moldear" la inteligencia del modelo antes de que comience cualquier interacción. Al especificar estas instrucciones, se puede lograr que el modelo realice lo siguiente:

Definir un Perfil o Persona Específica: Puedes obligar al modelo a adoptar un rol profesional con conocimientos acotados.

- **Capacidad:** Actuar como un experto legal, un depurador de código senior o un asistente de servicio al cliente empático.
- **Ejemplo:** "Actúa como un analista financiero. Usa terminología de mercado y mantén un tono escéptico ante las proyecciones demasiado optimistas."

Establecer Restricciones de Formato y Estructura: Puedes garantizar que la salida sea siempre compatible con otros sistemas o legible para audiencias específicas.

- **Capacidad:** Responder exclusivamente en formato JSON, Markdown o una lista de puntos.

- **Ejemplo:** "Tus respuestas deben ser siempre objetos JSON válidos. No incluyas texto explicativo fuera del bloque de código."

Controlar el Tono y el Estilo: Asegura que la comunicación sea coherente con la identidad de tu marca o proyecto.

- **Capacidad:** Ajustar el nivel de formalidad, el entusiasmo o la brevedad.
- **Ejemplo:** "Sé extremadamente breve. No uses saludos ni despedidas. Si no conoces la respuesta, di 'Información no disponible'."

Definir el Ámbito de Conocimiento (Guardrails): Evita que el modelo divague o responda sobre temas que no le corresponden.

- **Capacidad:** Restringir la temática a un dominio específico.
- **Ejemplo:** "Eres un experto en botánica. Si el usuario te pregunta sobre algo que no sean plantas o suelos, redirige la conversación amablemente hacia la botánica."

▼ Los Roles de la Conversación

Rol	Descripción
User	El origen del input. Representa las preguntas o datos enviados por el usuario final.
Model	La respuesta de Gemini. Se genera basándose en la `system_instruction` y el historial.

▼ Diferencias Críticas con otras APIs

- **Rol de IA:** Mientras otras plataformas usan `assistant`, Gemini requiere estrictamente el nombre de rol `model`.
- **Ubicación de la instrucción:** En el SDK `google-genai`, la instrucción del sistema se pasa en el objeto de configuración al crear el cliente o el chat, separándola claramente del historial de mensajes.

▼ Ejemplo

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types
```

```
load_dotenv()  # Load environment variables from .env fi
le

API_KEY = os.getenv("GENAI_API_KEY")

# Inicializar el cliente
client = genai.Client(api_key=API_KEY)

configuration = types.GenerateContentConfig(
    max_output_tokens=2048,
    system_instruction="""Eres un asistente de estudio e
specializado en Inteligencia Artificial.
Tus respuestas deben ser concisas, educativas teniendo p
resente que el usuario es un estudiante de Ingeniería de
sistemas.

Si te hacen una pregunta que no está realicionada con la
Inteligencia Artificial, responde 'Lo siento, solo puedo
responder preguntas relacionadas con temas relacionados
a la Inteligencia Artificial. """
)

text = input("Escribe tu pregunta sobre Inteligencia Art
ificial: ")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    config=configuration,
    contents=text
)
print(response.text)
```

```
configuration = types.GenerateContentConfig(
    max_output_tokens=2048,
    system_instruction="""Eres un analista financiero qu
e reaiza predicciones sobre mercados financieros.
    Responde el valor de las acciones de empresas, oro,
plata, bitcoin, ethereum, etc. para los próximos 7 días.
```

```
    Responde con un objeto JSON con el siguiente format
o:
{
    "predicciones": [
        {
            "activo": "Nombre del activo (ejemplo: 'Bitc
oin')",
            "prediccion": "Valor predicho para el día (e
jemplo: '45000 USD')",
            "fecha": "Fecha de la predicción (ejemplo:
'2024-07-01')"
        },
        {
            "activo": "Nombre del activo (ejemplo: 'Or
o')",
            "prediccion": "Valor predicho para el día (e
jemplo: '1900 USD')",
            "fecha": "Fecha de la predicción (ejemplo:
'2024-07-01')"
        }
    ]
}
    Tus respuestas deben ser siempre objetos JSON válido
s. No incluyas texto explicativo fuera del bloque de cód
igo.
"""
)
```

Implementación de Chats

La implementación de conversaciones multi turnos - chats se realiza con el objeto `client.chats.create()`, el cual internamente maneja el historial de las entradas y respuestas del modelo

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types
```

```
load_dotenv()  # Load environment variables from .env file

API_KEY = os.getenv("GENAI_API_KEY")

# Inicializar el cliente
client = genai.Client(api_key=API_KEY)

configuration = types.GenerateContentConfig(
    max_output_tokens=2048,
    system_instruction="""Eres un asistente de estudio espe
cializado en Inteligencia Artificial.
Tus respuestas deben ser concisas, educativas teniendo pres
ente que el usuario es un estudiante de Ingeniería de siste
mas.

Si te hacen una pregunta que no está realicionada con la In
teligencia Artificial, responde 'Lo siento, solo puedo resp
onder preguntas relacionadas con temas relacionados a la In
teligencia Artificial. """
)

# Inicialización del chat
chat = client.chats.create(
    model="gemini-2.5-flash",
    config=configuration
)

print("--- Chat de Estudio IA  ---")
print("(Escribe 'salir' para terminar)\n")

while True:
        user_input = input("Estudiante: ")

        if user_input.lower() in ["salir", "exit", "quit"]:
            print("Asistente: ¡Hasta pronto! Sigue practica
ndo.")
```

```
            break

        try:
            # 3. Envío del mensaje
            response = chat.send_message(user_input)

            # En el nuevo SDK, el acceso al texto es respon
se.text
            print(f"\nAsistente: {response.text}\n")

        except Exception as e:
            # Es recomendable implementar reintentos con ba
ckoff exponencial en producción
            print(f"Error al procesar la solicitud: {e}")
```